

RAG 3일 특강

Day 2 — 파이프라인 심화

강사명 · 날짜

오늘의 로드맵

- M2.1 청킹 전략 심화 / M2.2 임베딩 모델 비교
- Lab2.0 나만의 시나리오 선택 — 오늘부터는 각자 다른 문서로 실습
- Lab2.1 PGVector 연동 (내 시나리오에 적용)
- M2.3 검색 품질 향상 + Lab2.2 하이브리드 검색 (내 시나리오에 적용)
- M2.4 Re-ranking / M2.5 Query Transformation
- Lab2.3 Advisor 체인 커스터마이징 (내 시나리오에 적용)
- Lab2.4 RAG를 도구로 쓰기 (내 시나리오에 적용)
- M2.6 RAG 프롬프트 엔지니어링

M2.1 청킹 전략 심화

Day1 청킹 복습 및 한계 재확인

- Day1: 고정 크기(TokenTextSplitter)로만 잘랐음 — 오늘은 더 똑똑하게 자르는 법
- 어제 실습 문서(MCM-200 매뉴얼)도 chunk size 300으로 기계적으로 잘랐을 뿐, 조항 경계는 신경 안 씀

Fixed-size vs Recursive Splitting

고정 크기

구현 쉬움, 문맥 무시

vs

Recursive

문단/문장 경계를 우선 존중하며 재귀적으로 분할

- 예: 문단 구분자(\n\n)로 먼저 나누고, 그래도 너무 길면 문장 단위(.), 그래도 길면 단어 단위로 순차적으로 재귀 분할

실측: Fixed-size vs Recursive, 실제 이용약관으로 비교

- 실제 스타트업 이용약관(25개조)으로 비교: Fixed-size 34청크 vs Recursive 21청크
- Fixed-size 청크 하나: "[1]. 이 약관의 내용은..." — 마침표로 문장이 시작됨(토큰 경계가 문장 경계를 무시)
- Recursive 청크는 항상 문장이 끝나는 지점에서 잘림 — 위 문제가 사라짐

```
class RecursiveCharacterSplitter {  
    // 우선순위: 문장(". ") → 줄바꿈("\n") → 공백(" ")  
    // PDF에서 추출한 텍스트는 줄바꿈이 "시각적 줄바꿈"(word-wrap)일 뿐이라  
    // 문장부호를 최우선 구분자로 뒤야 함 — 실제로 겪은 버그  
    List<Document> split(Document doc) { /* 재귀적으로 구분자를 낮춰가며 분할 */ }  
}
```

Semantic Chunking

- 문장 임베딩 유사도가 급격히 떨어지는 지점을 자연스러운 경계로 판단해서 자름
- 절차: 문장 단위로 먼저 쪼갬 뒤 각 문장을 임베딩 → 인접 문장 간 코사인 유사도 계산 → 유사도가 급락하는 지점을 청크 경계로 채택
- 장점: 주제가 바뀌는 지점에서 자연스럽게 잘림 / 단점: 문장마다 임베딩 호출이 필요해 비용·시간이 더 듦

실측: 위키백과 "김치" 문서로 문장 간 유사도 확인

- "김치 (위키백과)" → "역사" 로 넘어가는 지점: 유사도 0.27 (주제 전환, 뚝 떨어짐)
- "지역별 김치" 안에서 "경상도: 콩잎김치..." → "제주도: 동지김치..." 사이: 유사도 0.79~0.83 (같은 나열 목록이라 계속 높게 유지)
- "영양가" 섹션 진입 지점("85.8g." → "교역 통계(2004년 기준)"): 유사도 0.33으로 급락
- threshold=0.7 기준 실제 생성된 청크 수: 28개

Sliding Window 청킹

- 청크를 겹치게 이동시키며 자르는 방식 — overlap의 확장판
- 예: chunk size 300, stride 150이면 각 청크가 절반씩 겹치면서 이동
- 장점: 경계 근처 정보 손실이 최소화됨 / 단점: 같은 내용이 여러 청크에 중복 저장되어 인덱스 크기 증가

실측: 위키백과 "제주도" 문서로 겹침 확인

- window=300자, stride=150자로 실제 실행 → 4개 청크 생성 (겹치지 않았다면 문서 길이상 2~3개였을 것)
- 청크1이 청크0의 중간 지점(150자 위치)부터 시작해서, 청크0에만 있던 문장이 청크1에도 그대로 다시 등장함
- 예: "기후는 온대 기후로 겨울에도 영하로 떨어지지 않는다"라는 문장이 청크0 후반부와 청크1 전반부에 중복으로 들어감

문서 구조 활용 청킹

- 마크다운 헤더, 표, 목록 단위로 자르면 의미 단위가 자연스럽게 보존됨
- 예: ## 헤더가 나올 때마다 새 청크 시작, 표는 통째로 하나의 청크로 유지(중간에 자르면 행/열 관계가 깨짐)
- 우리 실습 문서라면 "제N조" 단위가 바로 이 구조적 경계 역할

실측: "제N조" 경계로 실제 이용약관을 잘라보니

- 정규식 (제\d+조) 경계로 실제 이용약관(25개조) 분할 → 30개 청크 생성 (25개가 아님)
- 원인: 조항 본문 안에서 다른 조항을 참조하는 문장("제6조 8항의 규정은...")도 새 경계로 잘못 인식됨
- 구조 기반 청킹도 완벽하지 않다는 실제 증거 — 본문 안의 참조 표현까지 구조 마커로 오인할 수 있음

메타데이터 풍부화 전략

- 제목, 섹션, 페이지, 작성일을 청크마다 태깅 → 필터 검색(Lab2.1)의 기반이 됨
- 예: {"section": "제3조", "category": "환불정책", "page": 1} 같은 메타데이터를 각 청크에 붙임
- 이후 "환불 관련 조항만 검색해줘" 같은 필터링 검색이 가능해짐

실전 팁: 도메인별 권장 chunk size

매뉴얼/FAQ

200~400 토큰 (질문-답변이 짧고 독립적)

법률/계약 문서

500~800 토큰 (조항 하나가 통째로 들어가야 문맥이 유지됨)

코드 문서

함수/클래스 단위 (토큰 수보다 논리적 단위가 우선)

M2.2 임베딩 모델 비교

임베딩 모델 벤치마크 지표

- MTEB 리더보드 개념 — "어떤 임베딩 모델이 좋은지" 공인된 비교 기준이 있다는 것
- 검색, 분류, 클러스터링 등 여러 태스크에 걸쳐 점수를 매겨 종합 순위를 매김

OpenAI vs 오픈소스 임베딩

OpenAI(text-embedding-3)

품질 안정적, API 호출 비용 발생, 네트워크 필요,
데이터가 외부로 나감

VS

BGE / e5 / nomic 등 오픈소스

무료, 로컬 실행 가능(Ollama), 모델별 성능 편차가 큼

한국어 임베딩 실측 비교

- nomic-embed-text: "물통 용량" 유사 문장 = 0.67 / "저녁 메뉴" 무관 문장 = 0.78 (더 높음, 잘못된 결과)
- bge-m3: 유사 문장 = 0.62 / 무관 문장 = 0.44 (정상)
- search_query:/search_document: 같은 nomic 권장 접두어를 붙여도 결과는 그대로 실패

로컬(Ollama) 임베딩의 현실적 제약

- 속도: GPU 없는 노트북에서는 문서 수백 개 임베딩에 수 분 소요될 수 있음
- 정확도: 상용 API 대비 최신 오픈소스 모델도 일부 태스크에서 뒤처질 수 있음
- 메모리: bge-m3(1.2GB) 같은 모델을 여러 개 동시에 로드하면 메모리 압박

임베딩 차원 수와 저장 비용

- 차원이 클수록 표현력은 좋아지지만 저장/연산 비용도 커짐
- 예: 384차원 vs 1024차원 — 문서 100만 개 기준 저장 공간이 약 2.6배 차이
- bge-m3는 1024차원 — 표현력과 비용의 합리적 중간 지점

벡터DB 심화: 고차원의 함정

- "차원이 클수록 표현력이 좋다"고 방금 배웠는데, 그럼 차원을 무한정 키우면 안 될까?
- 정답: 안 됨 — curse of dimensionality(차원의 저주)라는 현상 때문
- 직관: 차원이 커질수록 아무 벡터 두 개를 뽑아도 서로 "거의 똑같이 멀어" 보이기 시작함

고차원의 함정을 코드로 실측하기

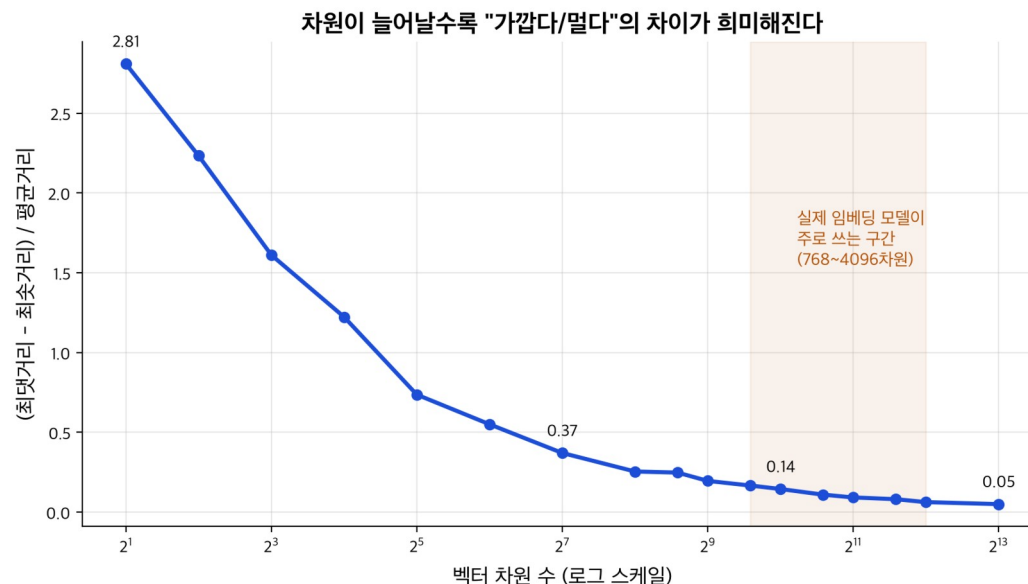
- 방법: 랜덤 벡터를 차원별로 2000개씩 생성해서, 기준점 하나와 나머지 벡터들 사이 거리의 "편차 비율"을 측정
- $(\text{최대거리} - \text{최소거리}) / \text{평균거리}$ — 이 값이 0에 가까울수록 "다 거기서 거기"라는 뜻
- Java·Python 양쪽에서 동일한 결과 → 특정 언어·라이브러리의 버그가 아니라 실재하는 현상

```
static double[] concentrationRatio(int dim, int n, long seed) {
    Random rng = new Random(seed);
    double[][] vectors = new double[n][dim];
    for (int i = 0; i < n; i++)
        for (int d = 0; d < dim; d++)
            vectors[i][d] = rng.nextGaussian();

    double max = Double.MIN_VALUE, min = Double.MAX_VALUE, sum = 0;
    for (int i = 1; i < n; i++) {
        double dist = euclidean(vectors[0], vectors[i]);
        max = Math.max(max, dist);
        min = Math.min(min, dist);
        sum += dist;
    }
    double mean = sum / (n - 1);
    return new double[]{max, min, mean, (max - min) / mean};
}
... (1줄 생략)
```

실측 결과: 차원이 커질수록 구별이 흐려진다

- 2차원: 비율 2.81 (거리 차이가 큼 → 구별 잘 됨)
- 128차원: 비율 0.36
- 1024차원(bge-m3): 비율 0.13
- 8192차원: 비율 0.05 (거의 다 거기서 거기)
- 실제 임베딩 모델들이 768~4096차원 구간에 몰려있는 이유와 정확히 일치



실전 시사점: 그래서 어떻게 해야 하나

- 무작정 고차원 임베딩을 고르는 게 능사가 아님 — 적정선이 있음
- Matryoshka 임베딩: 처음부터 "앞부분만 잘라 써도 성능이 크게 안 떨어지게" 학습시키는 최신 기법
- 실무 조언: 임베딩 차원은 벤치마크(MTEB) 성능과 저장/연산 비용을 같이 보고 결정

임베딩 모델 교체 시 주의점

- 모델을 바꾸면 기존 벡터와 비호환 → 반드시 전체 재인덱싱 필요
- 이유: 서로 다른 모델이 만든 벡터는 좌표계 자체가 다름 (같은 384차원이어도 의미 없음)

우리가 쓸 모델: bge-m3 (확정)

- 이유: 위 실측 데이터에서 한국어 의미 구분이 명확히 우수했음
- 이유 추가: 1024차원은 방금 본 "고차원의 함정" 실험 기준으로도 합리적인 지점 (표현력은 충분, 구별력 손실은 작음)
- 노트북 사양과 무관하게 전원 통일 → PGVector dimensions도 1024로 통일

M2.2B 벡터 DB 선택 가이드

벡터DB 6종 한눈에 비교

PGVector	PostgreSQL 확장(자체호스팅) — 기존 SQL 생태계(백업/모니터링/JOIN) 그대로 재활용, 중소규모에 적합
Qdrant	Rust 기반, 자체호스팅+매니지드 — 메모리 효율적, 메타데이터 필터링 특히 강력, 예산 제약 있는 중간 규모에 적합
Milvus	자체호스팅(오픈소스)+Zilliz Cloud 매니지드 — 수십억 벡터급 엔터프라이즈 확장성, 벤치마크에서 낮은 레이턴시로 자주 1위
Weaviate	자체호스팅+매니지드 — 하이브리드 검색(벡터+키워드)과 GraphQL API가 강점, 내장 벡터화 모듈 지원
Chroma	자체호스팅 전용(경량) — 설치가 가장 간단, 개발자 친화적, 프로토타입/소규모 서비스에 적합
Pinecone	완전 매니지드(서버리스)만 제공 — 운영 부담 최소, 멀티리전, 대신 자체호스팅 선택지가 없음

로컬 설치 명령 모음 (Docker)

- Pinecone은 매니지드 전용이라 로컬 Docker 이미지가 없음 — 클라우드 콘솔에서 인덱스를 만들고 API 키로 접근하는 방식

```
# PGVector (오늘 우리가 쓰는 것)
docker run -d -p 5432:5432 -e POSTGRES_PASSWORD=postgres pgvector/pgvector:pg16

# Qdrant
docker run -d -p 6333:6333 qdrant/qdrant

# Milvus (standalone — 실제 운영은 docker-compose 권장)
docker run -d -p 19530:19530 milvusdb/milvus:latest standalone

# Weaviate
docker run -d -p 8080:8080 \
  -e AUTHENTICATION_ANONYMOUS_ACCESS_ENABLED=true semitechnologies/weaviate

# Chroma
docker run -d -p 8000:8000 chromadb/chroma
```

Spring AI VectorStore 지원 목록 — 인터페이스 추상화, 다시 한번

- Spring AI가 공식 지원하는 VectorStore 구현체: PGVector, Chroma, Milvus, Pinecone, Qdrant, Redis, Weaviate 등 다수 (Elasticsearch, MongoDB Atlas, Neo4j, OpenSearch, Oracle 등도 포함)
- 오늘 아침 Lab2.1에서 SimpleVectorStore를 PgVectorStore로 바꿀 때 add()/similaritySearch() 코드가 그대로 재사용됐던 것 기억나시죠
- 이 목록에 있는 다른 DB로 바꿀 때도 원리는 동일 — VectorStore 인터페이스 뒤에서 의존성과 설정만 바뀜

선택 기준 체크리스트

우리 팀에 자체호스팅 운영을
감당할 SRE/DBA가 있는가?

없으면 Pinecone 같은 매니지드 우선 고려

이미 PostgreSQL 인프라를
운영 중인가?

있으면 PGVector로 추가 인프라 없이 바로 시작 가능

문서 규모가 수억~수십억
건까지 커질 수 있는가?

Milvus 같은 엔터프라이즈급 검토

하이브리드 검색(벡터+키워드
+필터)이 핵심 요구사항인가?

Weaviate·Qdrant가 강점

예산이 매우 제한적이고 빠른
프로토타입이 목표인가?

Chroma로 가장 빠르게 시작

왜 이 강의는 PGVector를 선택했는가

- 대부분 이미 PostgreSQL/SQL을 다뤄본 경험이 있어 학습 곡선이 낮음
- 별도 벡터DB 인프라 없이 기존 RDB에 확장(extension) 하나만 추가하면 됨
- HNSW 인덱스, 코사인 거리, 메타데이터 필터링, 영속성까지 오늘 실습에 필요한 기능을 전부 지원
- "일단 있는 인프라로 빠르게 검증"이 필요한 실무 상황과도 정확히 맞아떨어짐

Lab 2.0 — 오늘 하루 들고 갈 나만의 시나리오 고르기

- 오늘부터는 다 같이 커피메이커 매뉴얼 하나만 보는 게 아니라, 아래 8개 시나리오 중 하나를 배정받아(또는 골라서) 하루 종일 그 위에 기법을 하나씩 이식해나감
- 8개 시나리오에는 전부 실제 상용 RAG 서비스를 본뜬 것이고, 문서도 전부 실제 공개 자료 — 지어낸 것 없음
- 준비물 없음(문서는 이미 프로젝트에 포함되어 있음) — 각자 배정된 문서에 대한 질문 5~10개만 미리 생각해둘 것(정답을 아는 것 + 문서에 없는 것 섞어서)

시나리오 1 — 이커머스 CS봇

- 실제 서비스: 무신사·컬리·쿠팡 등 다수 이커머스가 운영하는 AI 고객상담 챗봇
- 실제로 하는 일: 배송/교환/환불 정책 자동 응답, 주문 상태 조회 연동, 해결 안 되면 상담원 연결로 에스컬레이션
- 문서: 커피메이커 매뉴얼(한국어, 1페이지, 4청크)
- 하루 계획 — Lab2.1: PGVector 인덱싱 + 정책 Q&A / Lab2.2: 오류코드·모델명 질문에 하이브리드 검색 / Lab2.3: 검색 품질 파이프라인 조립 / Lab2.4: "환불 가능 여부"를 도구로 노출해 조건 판단 자동화
- 실측: Q. "환불은 며칠 안에 가능해?" → A. "환불은 구매일로부터 14일 이내에만 가능합니다. ... 제품 결함으로 인한 환불은 구매일로부터 30일 이내에만 가능합니다."

시나리오 2 — 개발자 문서 Q&A봇

- 실제 서비스: Cursor Docs, Mintlify 기반 다수 SaaS(Stripe·Supabase 등)의 공식 문서 챗봇
- 실제로 하는 일: API 사용법을 코드 예제와 함께 답변, 에러 메시지 붙여넣으면 관련 트러블슈팅 문서 검색, 버전별 정보 구분
- 문서: arXiv 논문 "Agentic RAG Survey"(영어, 25페이지, 134청크)
- 하루 계획 — Lab2.1: 134청크 PGVector 인덱싱 / Lab2.2: 기술 용어·모델명 하이브리드 검색 / Lab2.3: 리랭커로 정확한 섹션 찾기 + 출처 표기 / Lab2.4: "코드 예제 찾기"를 별도 도구로 분리해 필요할 때만 호출
- 실측: 이 문서가 바로 오늘 리랭커 실습에서 순수 벡터검색 top-5와 리랭크 top-5가 0/5로 완전히 달랐던 그 논문 — 검색 품질 문제가 실제로 있는 문서라는 뜻

시나리오 3 — 학술 리서치 어시스턴트

- 실제 서비스: Elicit, Perplexity, Consensus
- 실제로 하는 일: 특정 주장에 대한 근거를 논문에서 찾아 인용과 함께 제시, 여러 논문 요약 비교, "이 연구의 한계는?" 같은 비판적 질문에도 답변
- 문서: arXiv 논문 "LLM Agent 평가 Survey"(영어, 25페이지, 138청크)
- 하루 계획 — Lab2.1: 138청크 PGVector 인덱싱 / Lab2.2: 저자명·연도 하이브리드 검색 / Lab2.3: 리랭커로 근거 문단 정밀 추출 + 비판적 질문 테스트 / Lab2.4: 일반 지식 질문과 논문 근거 질문을 구분해서 도구 호출 여부 판단

시나리오 4 — 약관·정책 Q&A봇

- 실제 서비스: 법률테크 서비스들의 약관/계약서 분석 도구
- 실제로 하는 일: "이 조항이 무슨 뜻이야?"를 쉬운 말로 설명, 특정 상황(예: 미성년자 가입)에 적용되는 조항 검색, 불리한 조항 하이라이트
- 문서: 실제 스타트업(미디어레시피) 이용약관 원문(한국어, 5페이지, 49청크)
- 하루 계획 — Lab2.1: 49청크 PGVector 인덱싱 + 조항 카테고리 메타데이터 태깅 / Lab2.2: "미성년자"·"환불" 키워드 하이브리드 검색 / Lab2.3: 리랭커로 정확한 조항 우선 검색 / Lab2.4: 조항 번호까지 정확히 인용하도록 도구·프롬프트 설계

시나리오 5 — 사내 위키 검색봇

- 실제 서비스: Notion AI, Glean, Guru
- 실제로 하는 일: 여러 사내 문서를 통합 검색해서 자연어 질문에 답변, 출처 문서 링크 제공, 오래된 정보와 최신 정보 구분
- 문서: 위키백과 "국민건강보험"(한국어)
- 하루 계획 — Lab2.1: PGVector 인덱싱 / Lab2.2: 연도·비율 숫자 정확 검색에 하이브리드 필수 적용 / Lab2.3: 리랭커로 정밀도 개선 / Lab2.4: 답변에 출처 문서까지 표기하도록 도구화

시나리오 6 — 여행 추천봇

- 실제 서비스: 야놀자·여기어때의 AI 여행 추천, 마이리얼트립 AI 가이드
- 실제로 하는 일: 목적지 정보(교통·명소·기후) 질문 응답, 여행 코스 추천, 실시간 정보(날씨·혼잡도)와 결합해서 답변 조정
- 문서: 위키백과 "제주도"(한국어)
- 하루 계획 — Lab2.1: PGVector 인덱싱 / Lab2.2: 지명 고유명사 하이브리드 검색 / Lab2.3: 리랭커 적용 / Lab2.4: (도전) 실시간 날씨 API를 별도 도구로 추가 연결

시나리오 7 — 레시피/음식 지식봇

- 실제 서비스: 만개의레시피, 해먹남녀 등 레시피 앱의 AI 검색
- 실제로 하는 일: 재료 기반 레시피 추천, 조리법 단계별 안내, 대체 재료 제안
- 문서: 위키백과 "김치"(한국어, 6청크)
- 하루 계획 — Lab2.1: PGVector 인덱싱 + 지역별 메타데이터 태깅 / Lab2.2: 지역명 하이브리드 검색 / Lab2.3: 리랭커 적용 / Lab2.4: "냉장고 재료로 뭐 만들 수 있어?" 같은 역방향 질문 도구화 시도

시나리오 8 — 오픈소스 라이브러리 Q&A봇

- 실제 서비스: 많은 오픈소스 프로젝트가 운영하는 공식 문서/Discord 봇
- 실제로 하는 일: 설치/설정 질문 응답, 버전 호환성 질문, 공식 예제 코드 제공
- 문서: Spring AI 공식 GitHub README(영어)
- 하루 계획 — Lab2.1: PGVector 인덱싱(문서가 짧아 가장 빨리 완주 가능) / Lab2.2: 버전 번호 하이브리드 검색 / Lab2.3: 리랭커 적용 / Lab2.4: 툴 vs 프롬프트바인딩까지 전부 완성 → 시간 남으면 다른 시나리오 문서로도 실험

실측: 서로 다른 문서에서도 같은 코드가 그대로 동작함

- 검증 1: 커피메이커 매뉴얼(한국어, 1페이지, 6개 청크) — PGVector 인덱싱+검색+재시작 영속성까지 확인
- 검증 2: arXiv 논문(영어, 25페이지, 134개 청크) — 같은 PagePdfDocumentReader/TokenTextSplitter/PgVectorStore 코드로 인덱싱+리랭크까지 확인
- 검증 3~8: 나머지 6개 시나리오 문서도 전부 Load+Split 정상 동작 확인 (스타트업 이용약관 5p/49청크, 위키백과 3종, Spring AI README 등)
- 문서의 언어·길이·형식이 완전히 달라도 파이프라인 코드는 단 한 줄도 안 바뀜

Lab 2.0 — 정답 세트까지 준비하기

- 질문 5~10개를 준비하는 데서 끝내지 말고, 질문마다 "정답이 있는 위치"와 "기대 답변 요약"까지 텍스트 파일로 적어둘 것
- 형식 예: Q. 환불은 며칠 안에 가능해? | 위치: 제3조 | 기대 답변: 구매일로부터 14일 이내(제품 결함은 30일 이내)
- 문서에 없는 내용을 묻는 질문은 정답 칸에 "문서에 없음"이라고 명시 — 이후 "모른다고 정직하게 답하는지" 채점 기준이 됨
- 이 파일이 오늘 하루 Lab2.1~2.4 전부와 마무리 리포트에서 쓰는 채점 기준지 — 지금 5분만 더 써두면 오후 내내 편해짐

Lab 2.1 — PGVector 연동

- 목표: Day1의 인메모리 SimpleVectorStore를, 방금 고른 여러분의 시나리오 문서를 담을 진짜 DB(PGVector)로 교체
- 왜 필요한가: SimpleVectorStore는 앱을 재시작하면 데이터가 전부 날아감 — 실서비스에서는 쓸 수 없음

Docker Compose로 PGVector 기동

- pgvector/pgvector:pg16 이미지는 PostgreSQL 16에 pgvector 확장이 이미 설치되어 있는 공식 이미지
- 실행: docker compose up -d 한 줄이면 로컬에 벡터 DB가 뜸

```
services:
  pgvector:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_DB: ragdb
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports: ["5432:5432"]
```

의존성 & application.yml

- index-type: HNSW는 어제 M1.3에서 배운 그 근사 최근접 이웃(ANN) 인덱스
- distance-type을 COSINE_DISTANCE로 지정 — 어제 직접 구현했던 코사인 유사도를 DB가 대신 계산해줌
- dimensions: 1024는 반드시 bge-m3의 실제 출력 차원과 일치해야 함(Lab1.1에서 확인한 숫자)

```
org.springframework.ai:spring-ai-starter-vector-store-pgvector
```

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ragdb
  ai:
    vectorstore.pgvector:
      index-type: HNSW
      distance-type: COSINE_DISTANCE
      dimensions: 1024    # bge-m3 기준, Lab1.1에서 확인한 값
```

테이블/HNSW 인덱스 자동 생성 확인

- DB 클라이언트로 vector_store 테이블과 인덱스가 자동 생성된 것을 직접 확인
- Spring AI가 애플리케이션 기동 시점에 필요한 테이블/인덱스를 알아서 생성해줌(schema-init)
- 확인 방법: psql이나 DBeaver로 접속해서 \d vector_store 실행

SimpleVectorStore → PgVectorStore 교체

- Day1 코드에서 빈 타입만 교체, add()/similaritySearch() 호출부는 그대로
 - 실제로 바뀌는 코드: SimpleVectorStore.builder(embeddingModel).build() → 의존성 주입만으로 PgVectorStore 빈이 자동 구성됨
 - 관찰 포인트: VectorStore 인터페이스 추상화 덕분에 교체가 이렇게 쉬움

재인덱싱 + DB에서 벡터 직접 확인

- SQL로 vector_store 테이블 조회해서 저장된 벡터 값 직접 확인
 - 예: `SELECT id, content, embedding FROM vector_store LIMIT 1;`로 실제 1024차원 벡터가 저장된 것을 확인

디버깅: dimensions 값을 일부러 틀리게 설정해보기

- application.yml의 dimensions: 1024를 768로 바꾸고 테이블을 새로 만들어서 실행해볼 것
- 실제로 뜨는 에러 메시지를 읽고, 왜 이 에러가 나는지 원인을 스스로 설명해볼 것
- 정답 코드 없음 — 에러 메시지를 원래대로(1024) 되돌려서 정상 동작까지 직접 복구할 것

실측: 재시작해도 데이터가 살아있는가

- 1회차 실행: 인덱싱 진행, "생성된 청크 수: 6"
- 2회차 실행(앱 재시작, 재인덱싱 없이): "시작 시점 PGVector에 이미 저장된 문서 수: 6" — 그대로 살아있음
- SimpleVectorStore였다면 2회차에서 0부터 다시 시작했을 것

메타데이터 필터 검색 구현

- 특정 섹션/카테고리로 필터링해서 검색 범위를 좁히는 코드
- 예: `SearchRequest.builder().query(q).filterExpression("category == 'refund']").build()`처럼 메타데이터 조건을 검색에 결합
- M2.1에서 태깅해둔 메타데이터(섹션, 카테고리)가 여기서 실제로 쓰임

SimpleVectorStore vs PGVector

SimpleVectorStore

빠른 시작, 휘발성(재시작하면 사라짐), 메타데이터 필터링 제한적

VS

PGVector

영속성, 확장성(수백만 건도 처리), 메타데이터 필터링 강력, 기존 SQL 생태계(백업, 모니터링 도구) 그대로 활용

트러블슈팅 & 정리

- Docker 포트 충돌 → docker-compose.yml에서 포트를 5433:5432처럼 왼쪽만 바꾸고 application.yml의 url도 맞춰서 수정
- pgvector 확장 미설치 → 공식 이미지(pgvector/pgvector)를 썼는지 확인, 직접 만든 이미지라면 CREATE EXTENSION vector; 수동 실행

도전 과제: 카테고리 필터 검색 구현

- 지금까지는 문서를 카테고리 구분 없이 통째로 저장 — 이번엔 메타데이터로 태깅해서 필터링할 것
- 목표: 여러분의 시나리오 문서를 내용상 자연스러운 카테고리 2~3개로 나눠서 태깅하고, 특정 카테고리 안에서만 검색되게 만들기 (예: 매뉴얼이면 "사용법/보증정책/오류코드", 논문이면 "서론/방법론/결론")
- 힌트: Document 생성 시 metadata Map에 category 같은 키를 채워 넣기
- 힌트: SearchRequest의 filterExpression을 이용하면 특정 메타데이터 값으로 검색 범위를 좁힐 수 있음
- 정답 코드 없음 — 한 카테고리만으로 필터링했을 때 실제로 다른 카테고리 내용이 안 섞여 나오는지 직접 확인할 것

도전 과제: 청킹 기법 3종 비교 실측

- M2.1에서 배운 5개 청킹 기법(Fixed-size/Recursive/Semantic Chunking/Sliding Window/구조 기반) 중 최소 3개를 자기 시나리오 문서에 직접 적용
 - 참고 구현: rag-day2-demo의 lab21m1 패키지(RecursiveCharacterSplitter/SemanticChunker/SlidingWindowSplitter/StructureBasedSplitter)를 스터디해서 문서 경로만 자기 시나리오로 바꿔 실행
 - 구조 기반 청킹은 시나리오마다 경계 마커가 다름 — 약관은 "제N조", 논문/위키는 섹션 제목이나 마크다운 헤더이므로 정규식/파서를 직접 자기 문서에 맞게 조정할 것 (D2-07B에서 본 참조 표현 오탐 함정도 그대로 만날 수 있음)
 - 각 기법으로 만든 청크 수 + Lab2.0 질문 세트 기준 hit rate(정답 위치가 담긴 청크가 검색 top-5에 실제로 들어오는지)를 비교표로 기록
 - 주의: Semantic Chunking은 문장 수만큼 임베딩 호출이 필요함 — 25페이지짜리 논문 시나리오처럼 문장이 많은 문서는 청킹 한 번에 몇 분씩 걸릴 수 있으니 미리 감안하고 시작할 것
 - 정답 코드 없음 — 청킹 방식이 바뀌면 어떤 질문의 hit/miss가 바뀌는지 직접 관찰할 것

M2.3 검색 품질 향상

Top-K가 너무 작거나 클 때

- 너무 작으면($K=1\sim 2$) 정답이 근처에 있어도 놓칠 수 있음
- 너무 크면($K=50+$) 노이즈 증가 + 프롬프트가 비대해져서 비용·속도 저하

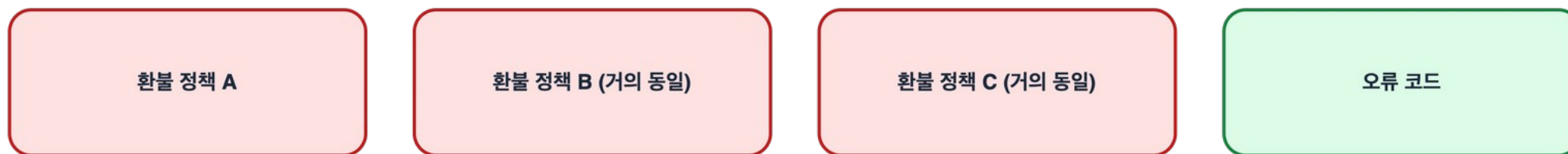
Similarity Threshold 설정

- 일정 유사도 이하 결과는 아예 버려서 무관한 결과를 걸러냄
- 예: threshold 0.5로 설정하면 유사도 0.3짜리 관련 없는 문서는 Top-K 안에 들어도 걸러짐

MMR (Maximal Marginal Relevance)

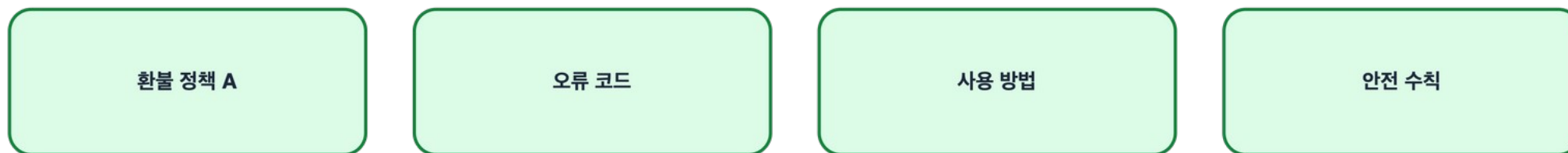
- 관련성은 높이면서 서로 비슷한 중복 결과는 줄이는 기법
- 예: Top-5를 뽑을 때 이미 뽑힌 문서와 너무 비슷한 문서는 순위가 낮아도 건너뛰고 다른 관점의 문서를 채택

일반 Top-K (유사도 순, 중복 많음)



환불 정책 관련 문서만 3개, 실제로 다른 정보(오류 코드)는 밀려남

MMR 적용 (관련성 + 다양성)



겹치는 내용은 줄이고, 서로 다른 주제를 고르게 포함

순수 벡터 검색의 한계

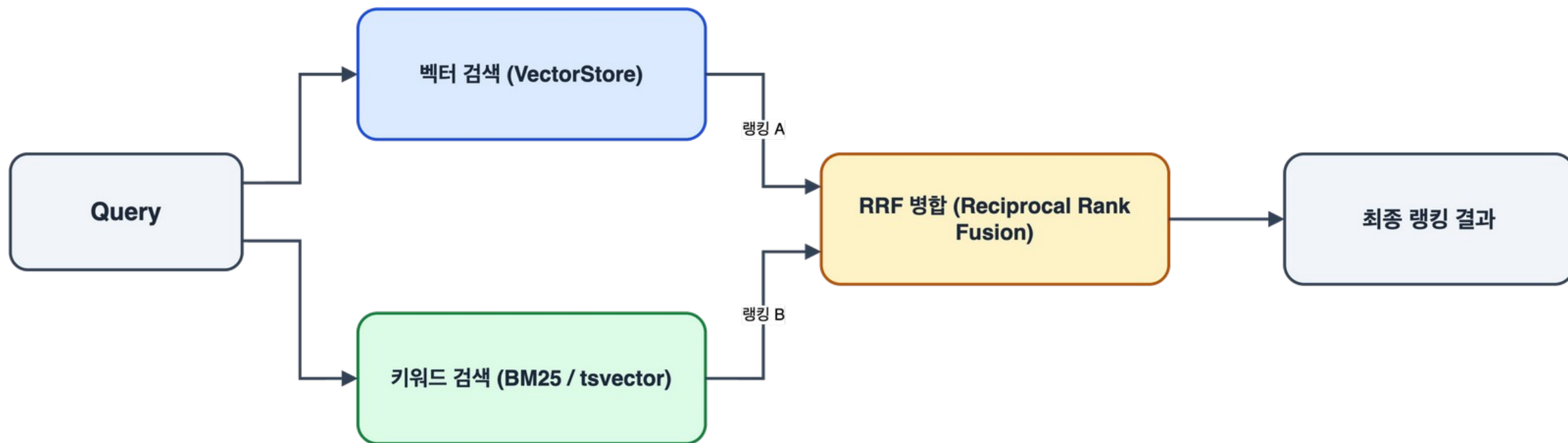
- 고유명사, 정확한 코드/모델명 등은 벡터 검색이 오히려 취약할 수 있음
- 예: "MCM-200"이라는 모델명은 의미상 별다른 뉘앙스가 없어서 임베딩이 다른 제품명과 잘 구분 못 할 수 있음

BM25 키워드 검색 원리

- 단어 빈도 기반 전통적 검색 방식 — 고유명사에 강함
- 문서에 특정 단어가 얼마나 자주(하지만 전체 문서 집합에서는 드물게) 등장하는지로 점수를 매김

하이브리드 검색 & RRF

- 벡터 검색 + 키워드 검색 결과를 RRF(Reciprocal Rank Fusion)로 병합
- RRF 공식: 각 결과의 순위(rank)에 대해 $1 / (k + \text{rank})$ 를 점수로 부여하고 두 리스트의 점수를 합산 (보통 $k=60$)
- 장점: 벡터 검색의 "의미 이해"와 키워드 검색의 "정확한 매칭"을 동시에 얻음



정리

- 오늘 배운 것: Top-K/threshold 튜닝 → MMR로 다양성 확보 → 하이브리드 검색으로 정확도까지

Lab 2.2 — 하이브리드 검색 구현

- 목표: 벡터 검색 + 키워드 검색을 합쳐서 여러분 문서의 고유명사 질의에도 강한 검색 만들기
- Lab2.0에서 준비한 질문 중 고유명사·숫자가 들어간 것을 골라서 테스트에 사용

PGVector tsvector 기반 키워드 검색

- to_tsvector가 텍스트를 검색 가능한 형태로 변환, plainto_tsquery가 검색어를 같은 형태로 변환해서 매칭
- 'simple' 설정을 쓴 이유: 한국어 형태소 분석 사전이 기본 내장되어 있지 않아 일단 단순 매칭으로 시작(정교화는 스트레치 과제)
- 아래는 기존 Day1~2 벡터 검색 코드를 그대로 재사용 (vectorStore.similaritySearch(...))

```
                List<Document> keywordResults = jdbcTemplate.query("""
SELECT id, content, metadata FROM vector_store
WHERE to_tsvector('simple', content)
      @@ plainto_tsquery('simple', ?)
LIMIT 20
""", documentRowMapper, keyword);
```

RRF로 두 결과 병합

- 두 리스트 모두 순회하면서 각 문서 ID에 $1/(k+\text{rank}+1)$ 점수를 더함 → 두 리스트 모두에서 상위권이면 점수가 두 배로 쌓임
- $k=60$ 은 RRF 논문에서 흔히 쓰이는 관행적인 값

```
Map<String, Double> rrfScore = new HashMap<>();  
  
int k = 60;  
// vectorResults, keywordResults 각각의 rank로 점수 누적  
rrfScore.merge(id, 1.0 / (k + rank + 1), Double::sum);
```

순수 벡터검색 vs 하이브리드 검색 비교

- 강사 예시(커피메이커 매뉴얼): 고유명사 "E2"/모델명 "MCM-200" 포함 질의로 테스트 → 하이브리드가 확실히 더 정확한 결과를 상위에 올림
- 여러분 차례: Lab2.0에서 준비한 질문 중 고유명사·숫자 포함 질문으로 자기 문서에 똑같이 재현

정리

- 순수 벡터 검색만으로는 부족한 지점을 실제로 확인

도전 과제: 가중치 있는 하이브리드 검색

- 지금 RRF는 벡터 검색과 키워드 검색에 동일한 가중치를 줌 — 이번엔 가중치를 다르게 줄 수 있도록 수정할 것
- 목표: 벡터 70% / 키워드 30% 처럼 비율을 파라미터로 조정 가능하게 RRF 공식 수정
- 힌트: 현재 `rrfScore.merge(id, 1.0 / (k + rank + 1), Double::sum)` 부분에 가중치를 곱하는 항을 추가
- 힌트: 비율을 0:100부터 100:0까지 바꿔가며, 여러분 문서의 고유명사 포함 질문 결과가 어떻게 달라지는지 표로 기록
- 정답 코드 없음 — 여러분의 시나리오 문서에 가장 잘 맞는 비율을 실험으로 찾아서 근거와 함께 제시할 것

도전 과제: 3-way 비교 자동화

- 순수 벡터 검색 / 순수 키워드 검색(tsvector) / 하이브리드(RRF) 3가지를 Lab2.0 질문 세트 "전체"에 대해 자동으로 돌리는 코드를 직접 작성
- 질문 목록을 순회하며 각 방식의 top-5 안에 정답 청크가 들어있는지(hit/miss)를 자동 판정해서 표로 출력
- 힌트: Lab2.0에서 적어둔 "정답 위치/기대 답변"을 검색 결과의 메타데이터·내용과 문자열로 비교하는 정도면 충분함 (엄밀한 자동 평가는 Day3에서 배움)
- 고유명사·숫자가 들어간 질문에서 세 방식의 hit/miss가 실제로 갈리는지 확인
- 정답 코드 없음 — 지금까지 짠 검색 코드를 반복문으로 감싸는 정도면 충분함

M2.4 Re-ranking

검색 후 재정렬이 또 필요한 이유

- Bi-encoder(임베딩) 검색은 질문/문서를 따로 벡터화 — 정밀도에 한계
- 질문과 문서를 독립적으로 벡터화하기 때문에, 둘 사이의 미묘한 상호작용을 놓칠 수 있음

Cross-encoder 원리

- 질문+문서를 함께 모델에 넣어서 정밀하게 관련도를 채점
- 질문과 문서를 하나의 입력으로 이어붙여서 모델에 넣기 때문에 둘 사이의 상호작용을 직접 학습

Bi-encoder vs Cross-encoder

Bi-encoder

빠름, 사전 인덱싱 가능, 정밀도 낮음

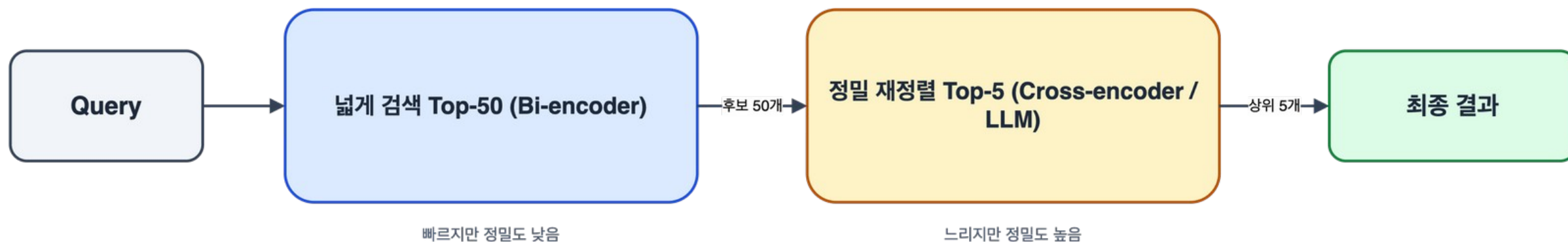
vs

Cross-encoder

느림(질문마다 재계산), 정밀도 높음

2단계 검색 패턴

- 넓게 검색(Top-50, Bi-encoder) → 정밀 재정렬(Top-5, Cross-encoder)
- 1단계에서 속도를 살려 넓게 후보를 걸러내고, 2단계에서 그 소수 후보에만 비싼 연산을 집중



Spring AI에서 Rerank 연동 방법

- 내장 Advisor 없음 → 커스텀 DocumentRetriever로 감싸서 구현
- 진짜 Cross-encoder 모델(BAAI/bge-reranker 등)은 별도 인프라가 필요해서, 이 강의에서는 이미 갖고 있는 LLM을 채점기로 재활용하는 방식으로 대체

정리

- Bi-encoder로 빠르게, Cross-encoder(또는 LLM)로 정밀하게 — 두 단계 조합이 핵심

M2.5 Query Transformation

사용자 원본 질문의 한계

- 짧고 모호한 질문은 검색에 최적이지 않음
- 예: "그거 얼마야?"처럼 대명사만 있는 질문은 임베딩해도 검색에 필요한 정보가 부족

Query Expansion

- 동의어/관련어를 추가해서 검색 커버리지를 넓힘
- 예: "환불"이라는 질문에 "반품", "취소", "교환" 같은 관련어를 추가해서 검색 범위를 넓힘

HyDE (Hypothetical Document Embeddings)

- LLM으로 가상의 답변을 먼저 생성 → 그 답변으로 검색 (질문보다 답변이 문서와 더 비슷하다는 아이디어)
- 예: "환불 정책이 뭐야?"라는 질문 대신, LLM이 상상으로 써낸 가상의 답변 문단을 임베딩해서 검색에 사용

Multi-query Retrieval

- 질문을 여러 버전으로 재작성 → 각각 검색 후 결과 병합
- 예: "환불 기한이 며칠이야?" → "환불 가능 기간", "반품 마감일", "구매 후 환불 신청 기한" 등 3~4개 변형 질문으로 각각 검색

Spring AI QueryTransformer

- RewriteQueryTransformer는 LLM을 이용해 원본 질문을 검색에 더 적합한 형태로 자동으로 다듬어줌

```
        QueryTransformer queryTransformer = RewriteQueryTransformer.builder()  
        .chatClientBuilder(chatClientBuilder)  
        .build();  
Query transformedQuery = queryTransformer.transform(query);
```

정리

- Query Expansion, HyDE, Multi-query — 검색 전에 질문 자체를 다듬는 세 가지 접근

Lab 2.3 — Advisor 체인 커스터마이징

- 목표: Query Transform + Rerank + Retrieval을 하나의 파이프라인으로 조립
- 오늘 배운 개념을 실제 Advisor 체인 하나로 조립하는 종합 실습

전체 파이프라인 정리

- Query Transform → Retrieve → Rerank → Augment → Generate



RewriteQueryTransformer 적용

- 실제 코드로 질문 재작성기를 파이프라인에 연결
- 아까 M2.5에서 만든 QueryTransformer를 RetrievalAugmentationAdvisor의 `.queryTransformers(...)`에 그대로 끼워 넣음

실측: 리랭크 전후로 top-5가 얼마나 달라지나

- 실습 문서(arXiv 논문, 25페이지 → 134개 청크)로 실제 측정한 결과
- 순수 벡터 검색 top-5 vs 넓게 검색(top-20) 후 LlmReranker로 재선정한 top-5
- 결과: 두 top-5가 단 하나도 안 겹침 (0/5) — 벡터 유사도 1등이 LLM 재채점에서는 상위 5위 밖으로 밀려남
- 코드는 M2.4에서 이미 본 LlmReranker/RerankingDocumentRetriever 그대로 — 여기서는 그 코드를 실행한 실제 결과만 확인

RetrievalAugmentationAdvisor 최종 조립

- 지금까지 만든 QueryTransformer와 RerankingDocumentRetriever를 하나의 Advisor로 조립
- 이 Advisor를 Day1의 ChatClient에 붙이면 오늘 배운 모든 기법이 적용된 RAG 파이프라인 완성

```
Advisor advisor = RetrievalAugmentationAdvisor.builder()  
.queryTransformers(rewriteQueryTransformer)  
.documentRetriever(rerankingDocumentRetriever)  
.build();
```

통합 테스트 — Day1 실패 케이스 재검증

- Day1 Lab1.3에서 실패했던 질문을 다시 던져서 개선 여부 확인
 - 예: "이 제품 방수 등급이 IP68 맞지?"처럼 문서에 없는 내용을 물었을 때, 오늘 파이프라인이 얼마나 더 신중하게 반응하는지 비교

성능/응답시간 비교

- Day1 단순 버전 vs Day2 고도화 버전 — 응답 품질은 좋아지지만 응답 시간은 늘어남 (LLM 호출 횟수 증가)
 - 예: Day1은 LLM 호출 1회, Day2 버전은 쿼리 재작성 1회 + 재정렬 후보마다 1회씩 + 최종 답변 생성 1회로 호출 횟수가 크게 늘어남

정리

- 오늘 배운 모든 기법을 하나의 파이프라인으로 조립하는 경험을 함

Lab 2.4 — RAG를 도구로 쓰기

- 목표: "무조건 검색하는" 프롬프트 바인딩형과 "모델이 필요할 때만 검색하는" 툴 방식을 실제로 비교

프롬프트 바인딩 vs 툴로서의 RAG

프롬프트 바인딩(QuestionAnswerAdvisor)

질문과 무관해도 매번 검색 실행, "컨텍스트에 없으면 모른다고 하라"는 지시에 묶임

VS

툴로서의 RAG(`@Tool`)

검색을 LLM이 호출 여부를 스스로 판단하는 함수로 노출, 컨텍스트 유무와 상관없이 자기 지식으로 답할 자유가 있음

- @Tool 어노테이션 + ToolCallingAdvisor로 구현 (Spring AI 2.0, 2026-06 GA — 툴콜링이 Advisor 체인 안에 정식 편입됨)

검색을 @Tool로 노출하기

- 등록: chatClient.prompt().tools(new DocumentSearchTool(vectorStore)).user(question).call()

```
public class DocumentSearchTool {  
    private final VectorStore vectorStore;  
  
    @Tool(description = "문서에서 질문과 관련된 내용을 검색한다.")  
    public String searchDocument(@ToolParam(description = "검색할 질문") String query) {  
        var results = vectorStore.similaritySearch(  
            SearchRequest.builder().query(query).topK(3).build());  
        return results.stream().map(Document::getText).collect(...);  
    }  
}
```


실측: 무관한 질문에 대한 두 방식의 차이

- 강사 예시(커피메이커 매뉴얼): "대한민국의 수도는 어디야?" (문서와 전혀 무관, 누구나 아는 사실)
- (A) 프롬프트 바인딩: "정보가 부족하여 정밀한 답변을 줄 수 없습니다" — 뻔히 아는 사실도 답변을 거부함
- (B) 톨로서의 RAG: 도구는 호출했지만 컨텍스트에 없매이지 않아 "서울입니다"라고 정상 답변
- 여러분 차례: 자기 문서와 전혀 무관한 상식 질문 하나로 (A)/(B) 재현해볼 것

정리 — 언제 뭘 쓸까

- 프롬프트 바인딩: 챗봇의 목적이 "이 문서에 대해서만 답하는 것"으로 명확할 때 (예: 사내 정책 챗봇)
- 톨로서의 RAG: 일반 대화와 문서 기반 답변을 동시에 지원해야 하거나, 여러 도구 중 하나로 검색을 쓰고 싶을 때

도전 과제: 불필요한 검색 스킵을 로그로 확인하기

- Lab2.0 질문 세트에 "검색이 필요 없는 일반 상식·인사 질문" 2~3개를 추가 (예: "안녕", "RAG가 뭐야?")
 - 프롬프트 바인딩 방식(QuestionAnswerAdvisor)과 툴 방식(@Tool) 양쪽에 전체 질문 세트를 실행하고, 검색이 실제로 호출됐는지 로그로 확인
 - 기대: 프롬프트 바인딩은 매번 검색 호출, 툴 방식은 일반 상식 질문에서 검색을 스킵
 - 만약 실제로 안 그렇다면(툴 방식도 매번 검색한다면) 그 이유를 도구 설명(description)이나 시스템 프롬프트를 고쳐서 직접 원인을 찾아볼 것
 - 정답 코드 없음

M2.6 RAG 프롬프트 엔지니어링

컨텍스트 주입 템플릿 구조

- 시스템 프롬프트 안에 "다음 컨텍스트를 참고해서 답하라"는 지시와 실제 검색된 문서 내용을 함께 삽입
- 이 구조가 Day1부터 QuestionAnswerAdvisor가 내부적으로 자동으로 해주던 일

출처(Citation) 표기

- 문서 메타데이터(제목, 페이지)를 답변에 함께 표기하도록 프롬프트 지시
- 예: "답변 끝에 참고한 조항 번호를 [제3조]처럼 표기하라"고 프롬프트에 명시
- 아까 M2.1에서 메타데이터를 태깅해둔 게 여기서도 쓰임

"모르면 모른다"고 답하게 만들기

- 컨텍스트에 없는 내용은 추측하지 말라는 명시적 지시 추가
- 예: "주어진 컨텍스트에 답이 없으면 '문서에서 관련 정보를 찾을 수 없습니다'라고만 답하라"는 지시를 시스템 프롬프트에 명시
- Day1 Lab1.3에서 관찰했던 실패 케이스가 여기서 해결됨

Day2 마무리 — 오늘의 실측을 리포트로 정리

- 오늘 만든 비교표들을 한 표로 합치기: 청킹 기법별 hit rate(Lab2.1), 검색 방식별 hit rate(Lab2.2), 검색 스킵 로그(Lab2.4)
- 표 아래에 발견한 것을 한 문단씩 정리: "내 시나리오 문서에서는 어떤 기법이 실제로 효과가 있었는가, 왜 그런 것 같은가"
- 발표 없음 — 강사가 자리마다 순회하며 화면으로 직접 확인
- 이 리포트는 내일 Day3 Lab3.1 평가 실습에서 그대로 재사용됨